

Tópico de Aplicaciones Web

Practica 4 De la capa de dominio a una API HTTP

Unidad I | Plan 2023

Documento de entrega

P04_00000240694

Alumno: Emmanuel Riveros

Matrícula: 00000240694

Qué se hizo

La capa de dominio de la práctica 3 se expuso como una API REST con Express. No cambié las carpetas dominio, infra ni servicios: la capa HTTP solo valida datos que llegan por la red, llama al Service y convierte la entidad a un contrato público.

Archivos agregados

Archivo	Para qué sirve
contratos/prestamo.dto.ts	Define petición, respuesta, error y la conversión al DTO público.
src/http/validar.ts	Valida un cuerpo unknown y reúne todos los detalles antes de rechazarlo.
src/http/servidor.ts	Configura Express, las rutas GET y POST, estáticos, 404 y el middleware de errores.
practica4.http	Incluye las siete peticiones para usar con REST Client en VS Code.

Rutas y resultados

Petición	Resultado comprobado
GET /api/prestamos?libroId=LIB-0417	200 con [] al iniciar el servidor.
POST /api/prestamos	201 Created y Location: /api/prestamos/P-101.
POST con ejemplar 15 repetido	409 Conflict con el error de dominio traducido.
POST con JSON inválido	400 y lista de detalles de validación.
GET sin libroId	400 porque el parámetro es obligatorio.
DELETE /api/otra-cosa	404 Ruta no encontrada.
GET /	200 y carga el cliente HTML.

Cómo ejecutarlo

```
npm install
npm run dev
```

Capturas de las respuestas

Estas son las tres respuestas que se pidieron para la entrega. Se ejecutaron en el orden de practica4.http, manteniendo el mismo servidor para que el segundo POST detectara que el ejemplar 15 ya estaba ocupado.

1 Crear un préstamo

HTTP 201 Created

Location: /api/prestamos/P-101

```
{
  "folio": "P-101",
  "libroId": "LIB-0417",
  "ejemplares": [14, 15],
  "socioId": "S-001",
  "estado": "activo"
}
```

2 Repetir el ejemplar

HTTP 409 Conflict

POST /api/prestamos

```
{
  "error": "El ejemplar 15 ya esta prestado"
}
```

3 Validación en tiempo de ejecución

HTTP 400 Bad Request

POST /api/prestamos

```
{
  "error": "La peticion no cumple el contrato",
  "detalles": [
    "libroId debe ser un texto no vacio.",
    "socioId debe ser un texto no vacio.",
    "ejemplares debe ser un arreglo de numeros."
  ]
}
```

Respuestas de reflexión

1 Qué pasaría sin el manejo async de Express

Cada ruta async tendría que envolver su código en try/catch y pasar el error con next(error). Si no, un rechazo de una promesa no llegaría al middleware central y repetiríamos la misma lógica en todas las rutas.

2 Por qué el Service no lanza un 409

El Service entiende préstamos, no HTTP. Por eso lanza EjemplarPrestadoError cuando un ejemplar ya está ocupado. La capa HTTP interpreta ese error y lo convierte en 409 Conflict. Así la misma regla sirve para otro cliente sin meter códigos HTTP dentro del dominio.

3 Qué cambiaría para una app móvil

La app móvil puede consumir estas mismas rutas y contratos. No tendría que tocar el dominio, el Service ni el repositorio. Solo se crearía el cliente móvil y, si hicieran falta datos o rutas nuevas, se ajustarían el contrato y la capa HTTP.

Enlace del repositorio

Frontend Web Practicas: <https://github.com/EmmanuelRiveros/Frontend-Web-Practicas>

El README incluye las respuestas completas, y practica4.http conserva todas las peticiones de prueba para REST Client.